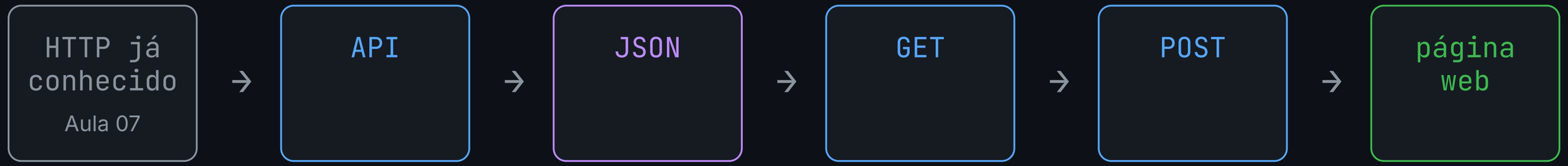
API Python: GET, POST e JSON

objetivo Consultar e enviar dados segundo o contrato de uma API.

// percurso da aula

O percurso de hoje vai da requisição à página web



Na Aula 07, identificamos as partes da requisição. Hoje vamos **produzir e consumir respostas de uma API** .

método

O que o cliente quer fazer

caminho

Qual recurso está sendo pedido

código de resposta

Como o servidor informa o resultado

Uma API permite que programas solicitem dados e ações

API, Application Programming Interface , é uma interface com operações definidas para que um programa interaja com outro.

exemplo no projeto

A página pede `/alertas` ; a API Python devolve os dados em JSON.

analogia

Um cardápio lista o que pode ser pedido. Quem pede não precisa entrar na cozinha.

API não é

um site

O site pode usar uma API, mas não é ela.

API não é

um banco de dados

A API pode consultar um banco, mas não é ele.

A página atua como cliente da API Python



Os papéis dependem da comunicação: **quem pede é cliente; quem atende é servidor**.

Na Aula 07, o navegador pedia um arquivo HTML. Agora a página pede **dados** à API.

Um endpoint combina método e caminho

endpoint 1

consultar a lista de alertas

`GET` `/alertas`

endpoint 2

enviar dados para criar um alerta

`POST` `/alertas`

O **caminho é o mesmo**; a operação muda com o **método**.

`GET` consulta; `POST` envia dados para criação neste exemplo.

O contrato descreve como usar cada operação

MÉTODO E CAMINHO	FINALIDADE	ENTRADA	RESPOSTA PRINCIPAL
`GET /alertas`	Listar todos os alertas	sem corpo	`200 OK` + lista JSON
`GET /alertas/1`	Consultar um alerta pelo ID	ID no caminho	`200 OK` + objeto JSON
`GET /metricas`	Consultar o resumo	sem corpo	`200 OK` + resumo JSON
`POST /alertas`	Criar um alerta	JSON com 3 campos	`201 Created` + alerta

O cliente pode ser escrito a partir do contrato, **sem conhecer a implementação interna** da API.

A resposta também faz parte do contrato

requisição

```
GET /alertas/1 HTTP/1.1
```

resposta

```
HTTP/1.1 200 OK
{ "id": 1, "tipo": "porta_incomum", ... }
```

requisição

```
GET /alertas/999 HTTP/1.1
```

resposta

```
HTTP/1.1 404 Not Found
{ "detail": "Alerta não encontrado" }
```

método | caminho | entrada | formato de saída | códigos

Juntos, esses itens compõem o **comportamento esperado** da operação.

// bloco 1 · API e contrato

A API do IDS começa com dados ilustrativos

coletar tráfego
futuro

→

analisar e detectar
futuro

→

expor resultados pela API
hoje · com dados de exemplo

→

apresentar na web
hoje · primeira interface

entra na prática de hoje

API Python, dados de exemplo e a primeira interface web.

fica para depois

Captura real de tráfego e regras de detecção.

4 min para as 10 questões

Quiz 1: questões 1–4

01 Qual é o papel da API neste exemplo?

A) Escolher a rede Wi-Fi.

B) Oferecer operações para outros programas.

C) Substituir o navegador.

02 Ao consultar `/alertas`, a página web atua como:

A) cliente.

B) roteador.

C) banco de dados.

03 Qual conjunto identifica uma operação do contrato?

A) Apenas o IP.

B) Método e caminho.

C) Apenas o código de resposta.

04 `GET /alertas` e `POST /alertas` têm:

A) sempre o mesmo efeito.

B) caminhos distintos.

C) o mesmo caminho e métodos distintos.

questões 5–7

Quiz 1: questões 5–7

05 Para consultar um alerta específico, qual caminho se encaixa no contrato?

A) `/alertas/1`

B) `/alertas/todos/novo`

C) `/metricas/alertas`

06 O cliente precisa conhecer o código interno que guarda os alertas?

A) Sim, para usar GET.

B) Não, precisa conhecer o contrato.

C) Sim, para ler JSON.

07 Um aluno diz que os alertas mostrados agora vieram de pacotes capturados. Qual resposta é correta?

A) Vieram do Chrome.

B) Vieram do DNS.

C) São dados ilustrativos.

questões 8–10

Quiz 1: questões 8–10

08 Se os exemplos estiverem apenas em memória e a API reiniciar, o que pode ocorrer?

- A) Os alertas criados na sessão se perdem.
- B) O método GET deixa de existir.
- C) O navegador vira servidor.

09 `GET /alertas/999` consulta um ID ausente. Qual resposta esperamos?

- A) `201`
- B) `404`
- C) `200` com o alerta 1

10 Que informação falta em uma descrição que só diz “acesse `/alertas`”?

- A) O nome do professor.
- B) O método e o comportamento esperado.
- C) A cor do navegador.

Devolutiva do bloco 1

01

API é interface entre programas.

02

Método + caminho identificam a operação.

03

Os dados de hoje são ilustrativos.

JSON representa os dados trocados pela API

JSON é um formato **textual** e **estruturado** para representar dados.

trecho JSON

```
{ "tipo": "porta_incomum" }
```

textual

Pode ser lido por pessoas.

estruturado

Programas em linguagens diferentes conseguem interpretar.

dados ilustrativos

Um objeto JSON reúne propriedades de um alerta

GET /alertas/1 · corpo

```
{
  "id": 1,
  "origem": "192.0.2.10",
  "tipo": "porta_incomum",
  "descricao": "Conexão observada em porta não esperada"
}
```

"id"

número

"origem" · "tipo" · "descricao"

textos

regra de sintaxe

Chaves e textos usam **aspas duplas**.

chaves { }

delimitam o objeto

dados ilustrativos

Uma lista JSON reúne vários objetos

GET /alertas · corpo (abreviado)

```
[
  { "id": 1, "tipo": "porta_incomum", ... },
  { "id": 2, "tipo": "tentativas_repetidas", ... }
]
```

`[ ]`
colchetes: a **lista**

`{ }`
chaves: cada **objeto**

`...`
campos omitidos só no slide

`GET /alertas` devolve uma **coleção**; `GET /alertas/1` devolve **um item**.

// bloco 2 · JSON

O tipo de dado influencia a interpretação

TRECHO	TIPO	NO MODELO DE ALERTA?
`"id": 1`	número	sim
`"tipo": "porta_incomum"`	texto (string)	sim
`"ativo": true`	booleano	não · só exemplo de sintaxe
`"detalhes": null`	valor nulo	não · só exemplo de sintaxe

`1` e `"1"` são diferentes: o primeiro é número; o segundo, texto.

dados ilustrativos

No POST, o cliente envia dados e a API atribui o ID

o cliente envia

```
{
  "origem": "203.0.113.8",
  "tipo": "acesso_suspeito",
  "descricao": "Acesso observado fora do padrão"
}
```

API →

a API devolve

```
{
  "id": 3,
  "origem": "203.0.113.8",
  "tipo": "acesso_suspeito",
  "descricao": "Acesso observado fora do padrão"
}
```

O cliente **não envia o ID** neste contrato. Quem atribui o `id: 3` é a API.

// bloco 2 · JSON

O cabeçalho informa que o corpo contém JSON

requisição POST

```
POST /alertas HTTP/1.1
Content-Type: application/json

{
  "origem": "203.0.113.8",
  "tipo": "acesso_suspeito",
  "descricao": "Acesso observado fora do padrão"
}
```

erro 1 · sintaxe

Chave sem aspas: não é JSON válido.

```
{ origem: "203.0.113.8" }
```

erro 2 · contrato

JSON válido, mas falta o campo `tipo`.

```
{ "origem": "203.0.113.8",
  "descricao": "..." }
```

4 min para as 10 questões

Quiz 2: questões 1–4

01 JSON serve para:

A) definir uma porta TCP.

B) representar dados estruturados.

C) instalar Python.

02 Em `"id": 1`, o valor é:

A) número.

B) texto.

C) lista.

03 Qual símbolo envolve um objeto JSON?

A) `[]`

B) `()`

C) `{}`

04 `GET /alertas` devolve vários alertas. Qual estrutura externa faz sentido?

A) lista `[]`

B) apenas uma string

C) um número

questões 5–7

Quiz 2: questões 5–7

05 Qual chave é obrigatória no corpo do POST deste contrato?

A) `senha`

B) `tipo`

C) `horario_atual`

06 Quem atribui o `id` do alerta criado?

A) A API.

B) O navegador por sorteio.

C) O cabeçalho HTTP.

07 Qual cabeçalho anuncia o formato do corpo enviado?

A) `Accept-Language`

B) `Host`

C) `Content-Type: application/json`

questões 8–10

Quiz 2: questões 8–10

08 Qual trecho segue a sintaxe JSON?

A) `{tipo: "teste"}`

B) `{"tipo": "teste"}`

C) `{'tipo': 'teste'}`

09 O corpo tem JSON válido, mas não contém `tipo`. O problema é:

A) contrato de entrada.

B) endereço IP.

C) porta ocupada.

10 A resposta de `GET /alertas/1` representa:

A) uma lista obrigatória de 100 itens.

B) o servidor inteiro.

C) um objeto de alerta.

// bloco 2 · JSON · devolutiva

Devolutiva do bloco 2

01

Objeto usa `{ }`; lista usa `[ ]`.

02

Chaves e textos usam aspas duplas.

03

O corpo do POST deve respeitar o contrato.

GET solicita a representação de um recurso

GET `/alertas`

Consultar **sem criar** um alerta: o GET pede uma cópia dos dados.

Aula 07

`GET /index.html` → o servidor devolve um arquivo HTML.

Aula 08

`GET /alertas` → a API devolve dados em JSON.

dados ilustrativos

GET /alertas devolve uma lista e 200 OK

requisição

```
GET /alertas HTTP/1.1
Accept: application/json
```

no DevTools

Network → clique em alertas

Status fica em Headers; o **corpo** , em Response ou Preview.

resposta

```
HTTP/1.1 200 OK
Content-Type: application/json

[
  { "id": 1, "tipo": "porta_incomum", ... },
  { "id": 2, "tipo": "tentativas_repetidas", ... }
]
```

dados ilustrativos

GET /alertas/1 seleciona um item pelo ID

GET /alertas/1

resposta · 200 OK


```
{
  "id": 1,
  "origem": "192.0.2.10",
  "tipo": "porta_incomum",
  "descricao": "Conexão observada em porta não esperada"
}
```


pense

O que muda se trocarmos /1 por /2?

// bloco 3 · GET e leitura da resposta

Um ID inexistente resulta em 404

GET /alertas/1

```
HTTP/1.1 200 OK
{ "id": 1, ... }
```

GET /alertas/999

```
HTTP/1.1 404 Not Found
{ "detail": "Alerta não encontrado" }
```

404 significa

O servidor **recebeu** o pedido e **respondeu** que o recurso não existe.

404 não significa

Que a conexão falhou ou que nada foi enviado.

resumo de dados ilustrativos

GET /metricas devolve um resumo dos dados

GET /metricas · 200 OK

```
{
  "total_alertas": 2,
  "por_tipo": {
    "porta_incomum": 1,
    "tentativas_repetidas": 1
  }
}
```

total_alertas

Quantos alertas existem agora.

por_tipo

Quantos alertas há de cada tipo.

depois do POST

`total_alertas` passa a 3 e surge `acesso_suspeito: 1`.

// bloco 3 · GET e leitura da resposta

O Network mostra a requisição e a resposta separadamente

01 Abra a página da API no Chrome.

02 Pressione **F12** e selecione **Network**.

03 Execute o GET (recarregue ou clique no botão).

04 Clique na requisição `alertas` na lista.

05 Observe **Headers**, **Status** e **Response/Preview**.

checklist

- [ ] método
- [ ] caminho
- [ ] status
- [ ] JSON recebido

4 min para as 10 questões

Quiz 3: questões 1–4

01 Qual requisição lista os alertas?

A) `GET /alertas`

B) `POST /metricas`

C) `GET /alertas/999`

02 Qual status corresponde à consulta bem-sucedida do alerta 1?

A) `404`

B) `200`

C) `201`

03 O trecho `/1` em `/alertas/1` identifica:

A) a porta.

B) a versão do HTTP.

C) o ID solicitado.

04 Ao receber `404` de `/alertas/999` , podemos concluir que:

A) houve uma resposta HTTP indicando ausência do recurso.

B) o computador desligou.

C) o navegador não enviou nada.

Quiz 3: questões 5–7

05 Antes de criar outro alerta, `/metricas` informa `total_alertas: 2`. O valor representa:

A) a quantidade ilustrativa de alertas armazenados.

B) a quantidade de redes capturadas.

C) a porta do serviço.

06 Onde localizar o código de resposta no DevTools?

A) Apenas no código-fonte HTML.

B) Nas informações da requisição em Network.

C) No endereço IP local.

07 Qual resultado representa melhor `GET /alertas`?

A) Um objeto obrigatório com apenas o alerta 1.

B) Um formulário HTML.

C) Uma lista JSON.

questões 8-10

Quiz 3: questões 8–10

08 Uma consulta GET deve criar um novo alerta neste contrato?

A) Sim, sempre.

B) Não.

C) Apenas se o Chrome estiver aberto.

09 O aluno vê `200` mas não sabe quais dados chegaram. O que deve inspecionar?

A) O corpo da resposta.

B) A resolução da tela.

C) O nome da pasta.

10 `GET /alertas/2` deve consultar:

A) todas as métricas.

B) o alerta de ID 2.

C) uma nova página de cadastro.

Devolutiva do bloco 3

01

Método e caminho mostram o **pedido**.

02

Status e corpo mostram o **resultado**.

03

404 é uma resposta, não falta de conexão.

POST /alertas envia dados para criar um alerta

requisição

```
POST /alertas HTTP/1.1
Content-Type: application/json

{ corpo JSON com o novo alerta }
```

método

POST: enviar dados

cabeçalho

o corpo é JSON

corpo

os dados do alerta

dados ilustrativos

O corpo do POST deve conter três campos

corpo enviado

```
{
  "origem": "203.0.113.8",
  "tipo": "acesso_suspeito",
  "descricao": "Acesso observado fora do padrão"
}
```

obrigatórios

`origem`, `tipo` e `descricao`

pode mudar

Os **valores**, para testar outros exemplos.

não pode mudar

Os **nomes das chaves**.

dados ilustrativos

Uma criação válida retorna 201 e o novo recurso

antes

IDs 1 e 2

POST →

resposta

```
HTTP/1.1 201 Created
{ "id": 3, "tipo": "acesso_suspeito",
... }
```

GET →

depois

IDs 1, 2 e 3

`201 Created` confirma a criação. Um novo `GET /alertas` **comprova** que o alerta está na lista.

A página consulta a API com fetch()

script da página

```
const  resposta = await fetch("/alertas");
if  (!resposta.ok) throw new Error("Falha na consulta");
const  alertas = await  resposta.json();
```

LINHA	O QUE FAZ
1	Envia `GET /alertas` e espera a resposta.
2	Se o status não for de sucesso, interrompe com um erro.
3	Interpreta o corpo JSON e guarda a lista em `alertas`.

Nesta demonstração, a página é servida pela **mesma origem** da API.

O formulário envia JSON com fetch()

1 · preparar

```
const  novoAlerta = { origem, tipo, descricao };
```

2 · enviar

```
fetch("/alertas", {
  method: "POST",
  headers: {"Content-Type": "application/json"},
  body: JSON.stringify(novoAlerta)
});
```

`JSON.stringify()`

Transforma o objeto JavaScript em **texto JSON** para o corpo.

depois do envio

Verificar `response.ok` e atualizar a lista.

// prática · 12 minutos

Os alertas usados nesta prática são ilustrativos.

Prática: consultar, criar e confirmar

01 Inicie a API-base preparada pelo professor.

02 Abra a página servida pela própria API.

03 Consulte a lista e encontre `GET /alertas` no Network.

04 Envie o formulário com um alerta de exemplo.

05 Confira `POST /alertas`, `201 Created` e o JSON retornado.

06 Atualize ou consulte a lista e confirme o novo item.

07 Registre uma dúvida ou um erro observado.

4 min para as 10 questões

Quiz 4: questões 1–4

01 Qual método o contrato usa para criar um alerta?

A) GET.

B) POST.

C) DELETE.

02 Qual status indica a criação válida neste exemplo?

A) `201`

B) `404`

C) `405`

03 Em que parte da requisição vão os campos do novo alerta?

A) Apenas no código de resposta.

B) No corpo JSON.

C) No rodapé da página.

04 O novo alerta enviado sem `id` retorna com `id: 3` porque:

A) a API atribui o identificador.

B) o Chrome converte o IP em ID.

C) `fetch()` escolhe o número.

Quiz 4: questões 5–7

05 Para que serve `JSON.stringify(novoAlerta)`?

- A) Abrir uma porta.
- B) Converter o objeto JavaScript em texto JSON.
- C) Criptografar a página.

06 Depois do POST válido, qual teste confirma a criação?

- A) `GET /alertas` contendo o item novo.
- B) Fechar o terminal.
- C) Trocar a extensão do HTML.

07 Por que a página verifica `response.ok`?

- A) Para distinguir respostas de sucesso de respostas com erro.
- B) Para descobrir o IP do aluno.
- C) Para criar o ID antes da API.

questões 8–10

Quiz 4: questões 8–10

08 O formulário omite `tipo`. Qual resultado é coerente com a validação adotada?

A) `201` com alerta completo.

B) `422` por entrada inválida.

C) `404` para o alerta 1.

09 O que `response.json()` faz após o GET?

A) Interpreta o corpo JSON da resposta.

B) Envia automaticamente outro POST.

C) Inicia o servidor Python.

10 Se a API reiniciar e usar somente memória, o alerta criado durante a prática:

A) fica garantido no disco.

B) vira um cabeçalho.

C) pode desaparecer.

Devolutiva do bloco 4

01

POST envia o **corpo**.

02

201 confirma a **criação**.

03

GET permite **conferir** o resultado.

// fechamento da aula 08

Entrega e fechamento da Aula 08

checklist

- [ ] API executando
- [ ] contrato inicial identificado
- [ ] GET observado
- [ ] POST testado
- [ ] página consumindo JSON

entrega

Primeiro contrato e evidência simples da integração web/API.

Exemplo de evidência: captura do Network com `POST /alertas` e `201`.

para a próxima aula

Como organizar essas operações para que o serviço continue compreensível quando o IDS crescer?

Aula 09 · 60 minutos · Investigar

Hoje vamos organizar e testar o serviço

retomada · Aula 08

A página já consome `GET /alertas` e envia `POST /alertas`.

pergunta de hoje

Como manter o serviço compreensível quando o IDS crescer?

01 Organizar os recursos da API.

02 Distinguir HTTP, REST e SOA.

03 Explicitar o contrato, incluindo os erros.

04 Testar casos positivos e negativos.

05 Revisar a arquitetura do IDS.

O recurso é aquilo que o cliente deseja consultar ou criar

coleção

alertas

todos os alertas

item

alerta 7

um alerta identificado

resumo

métricas

contagens calculadas

A resposta JSON é uma **representação** do recurso: uma “fotografia” do estado dele no momento da consulta.

Caminhos distintos representam coleção, item e resumo

CAMINHO	RECURSO	LEITURA
`/alertas`	coleção	todos os alertas
`/alertas/7`	item	o alerta de ID 7, se existir
`/metricas`	resumo	contagens calculadas a partir dos alertas

O `7` mostra apenas o **formato** do caminho. Nos dados iniciais, existem os IDs 1 e 2.

O método indica a operação aplicada ao recurso

MÉTODO	CAMINHO	OPERAÇÃO
GET	/alertas	consultar a coleção
GET	/alertas/1	consultar um item
POST	/alertas	criar um item na coleção

REST é um estilo arquitetural para sistemas distribuídos

REST (Representational State Transfer) orienta a organização da interação por **recursos** e por uma **interface uniforme**.

recurso

`/alertas/1`

GET →

representação recebida

```
{ "id": 1, "origem": "192.0.2.10", "tipo": "porta_incomum", ... }
```

REST não é

um sinônimo de JSON

REST não é

um protocolo que substitui o HTTP

Uma API com JSON não se torna automaticamente REST

JSON

descreve os **dados**

Como os valores são escritos no corpo.

REST

descreve a **arquitetura e a interação**

Como cliente e servidor se organizam e conversam.

Bons nomes de caminhos e respostas em JSON, isoladamente, **não demonstram** que todas as restrições REST foram adotadas.

dados ilustrativos

O IDS expõe alertas como recursos identificáveis

cliente

Página web

`GET /alertas`	→ consulta a coleção
`GET /alertas/1`	→ seleciona um alerta
`GET /metricas`	→ consulta o resumo

aplicação

Qual caminho você escolheria para examinar **somente o alerta 2**?

4 min para as 10 questões

Quiz 5: questões 1–4

01 Neste exemplo, `/alertas` identifica:

A) uma coleção de recursos.

B) a senha da API.

C) o protocolo TCP.

02 O JSON devolvido em `/alertas/1` é:

A) a porta do servidor.

B) uma representação do alerta.

C) o navegador inteiro.

03 Qual caminho indica um alerta individual?

A) `/metricas`

B) `/alertas`

C) `/alertas/1`

04 REST é:

A) um protocolo que substitui HTTP.

B) um estilo arquitetural.

C) uma sintaxe JSON.

questões 5–7

Quiz 5: questões 5–7

05 Retornar JSON, por si só, prova que uma API atende a todas as restrições REST?

A) Sim.

B) Apenas se houver POST.

C) Não.

06 O que combina com `GET /metricas`?

A) Consultar um resumo.

B) Criar um alerta.

C) Apagar o alerta 1.

07 Um aluno quer consultar somente o alerta 2. Qual requisição segue a organização mostrada?

A) `POST /alertas`

B) `GET /alertas/2`

C) `GET /metricas/2`

questões 8–10

Quiz 5: questões 8–10

08 A diferença entre `/alertas` e `/alertas/1` é:

A) coleção versus item identificado.

B) HTTP versus UDP.

C) cliente versus servidor.

09 O método em `POST /alertas` indica:

A) o formato da tela.

B) a ação solicitada sobre o recurso.

C) o endereço MAC.

10 No exemplo de `/alertas/7`, podemos afirmar que o alerta 7 existe?

A) Sim, o caminho o cria.

B) Sim, todo número é válido.

C) Não; o caminho descreve a consulta, que pode resultar em 404.

Devolutiva do bloco 5

01

Recurso é o que se consulta ou cria.

02

Caminho identifica; **método** indica a ação.

03

Representação é o JSON que volta.

SOA organiza capacidades em serviços com responsabilidades claras

Arquitetura orientada a serviços organiza funções disponibilizadas por **interfaces e contratos** .

serviço

faz uma parte do trabalho

interface

diz como pedir

contrato

diz o que entra e o que sai

HTTP, REST e SOA respondem a perguntas diferentes

TERMO	O QUE É	RESPONDE A	NA API DE ALERTAS
HTTP	protocolo de comunicação da web	Como as mensagens são trocadas?	`GET /alertas` e a resposta `200`
REST	estilo arquitetural	Como organizar a interação?	recursos `/alertas`, `/alertas/1`, `/metricas`
SOA	abordagem para organizar responsabilidades como serviços	Quem faz o quê?	API de alertas como serviço da página

atenção

SOA não é sinônimo de SOAP.

atenção

REST e SOA podem coexistir.

Coleta e detecção reais ainda não foram implementadas

O IDS pode separar coleta, análise, API e interface

coletor Python

observa dados de rede

futuro

módulo de análise

aplica regras

futuro

API

expõe os resultados

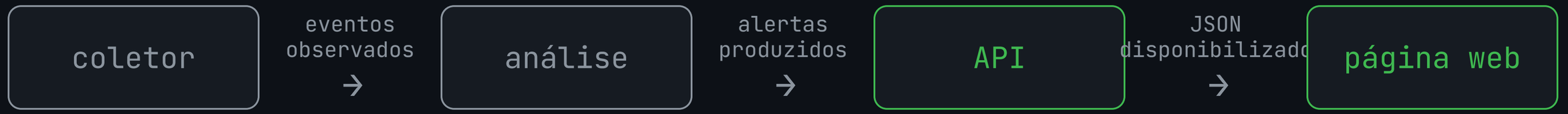
hoje · dados ilustrativos

página web

apresenta os resultados

hoje

Cada parte precisa de uma entrada e uma saída compreensíveis



A página transforma o JSON em **dados apresentados** ao usuário.

ainda em aberto

O formato definitivo entre coletor, análise e API será definido nas próximas etapas.

Um contrato permite trocar a implementação sem mudar o cliente

hoje

API com dados em memória

contrato preservado

`GET /alertas` → `200` + lista JSON

futuro

API recebe alertas do analisador

A página continua consultando `GET /alertas` sem perceber a troca.

mudança incompatível

Renomear campos ou caminhos exige revisar o contrato **e** o cliente.

A arquitetura do projeto começa pequena e pode evoluir

agora

01 API Python

02 exemplos em memória

03 página web

depois

01 coletor

02 análise e regras

03 alertas produzidos

04 integração com a API

pense

Que parte deve decidir se um evento gera alerta?

4 min para as 10 questões

Quiz 6: questões 1–4

01 HTTP é apresentado aqui como:

A) protocolo.

B) banco de dados.

C) regra de detecção.

02 SOA ajuda principalmente a:

A) configurar DNS.

B) organizar serviços e responsabilidades.

C) desenhar ícones.

03 REST é:

A) um arquivo Python obrigatório.

B) um endereço IP.

C) um estilo arquitetural.

04 Quem deve apresentar alertas ao usuário no projeto?

A) Interface web.

B) Coletor de pacotes.

C) Porta TCP.

Quiz 6: questões 5–7

05 Quem deve avaliar regras de detecção na arquitetura proposta?

A) O formulário HTML.

B) O módulo de análise.

C) O navegador enquanto renderiza CSS.

06 SOA significa necessariamente usar SOAP?

A) Sim.

B) Somente em Python.

C) Não.

07 REST e SOA são obrigatoriamente excludentes?

A) Não.

B) Sim, sempre.

C) Apenas se houver JSON.

Quiz 6: questões 8–10

08 Hoje os alertas estão em memória; amanhã vêm do analisador. O que a página precisa para continuar funcionando?

A) O mesmo computador físico.

B) Um contrato da API preservado ou atualizado de forma coordenada.

C) A mesma cor do slide.

09 Qual componente entrega JSON à página neste projeto?

A) API.

B) Roteador.

C) Terminal do aluno.

10 Já podemos afirmar que o IDS capturou tráfego real nesta aula?

A) Sim, porque aparece um IP no JSON.

B) Sim, porque existe um GET.

C) Não; os dados são ilustrativos.

Devolutiva do bloco 6

01

HTTP comunica.

02

REST orienta o desenho da interação.

03

SOA organiza serviços e responsabilidades.

// bloco 7 · contrato e erros

Um contrato completo também descreve entradas e falhas

MÉTODO	CAMINHO	ENTRADA	SUCESSO	ERROS ESPERADOS
GET	/alertas	sem corpo	200 lista	—
GET	/alertas/{id}	ID no caminho	200 objeto	404 ID ausente
GET	/metricas	sem corpo	200 resumo	405 se usar POST
POST	/alertas	JSON: origem, tipo, descricao	201 criado	422 campo obrigatório ausente

// bloco 7 · contrato e erros

O POST exige origem, tipo e descrição

válido → 201

```
{
  "origem": "203.0.113.8",
  "tipo": "acesso_suspeito",
  "descricao": "Acesso observado..."
}
```

sem "tipo" → 422

```
{
  "origem": "203.0.113.8",

  "descricao": "Acesso observado..."
}
```

No exemplo com **validação padrão do FastAPI**, campo obrigatório ausente resulta em `422 Unprocessable Content`.

Outras APIs podem usar `400` para entradas inválidas; o que vale é o que o contrato declara.

// bloco 7 · contrato e erros

O 404 informa que o recurso pedido não foi encontrado

```
GET /alertas/999

HTTP/1.1 404 Not Found
Content-Type: application/json

{ "detail": "Alerta não encontrado" }
```

404

A API **respondeu** que o item não existe.

erro de conexão

Não houve resposta HTTP do servidor.

// bloco 7 · contrato e erros

Entrada inválida e recurso ausente são problemas distintos

MÉTODO	CAMINHO	CONDIÇÃO	STATUS	INTERPRETAÇÃO
POST	/alertas	corpo sem tipo	422	o **pedido** está incompleto
GET	/alertas/999	ID inexistente	404	o **recurso** não existe

No 422, quem precisa corrigir é o **cliente**. No 404, o cliente pediu algo que **não está** lá.

Um caminho existente pode rejeitar outro método

`GET /metricas`

```
HTTP/1.1 200 OK
```

`POST /metricas`

```
HTTP/1.1 405 Method Not Allowed
```

405

o caminho **existe**, mas não aceita esse método

404

o recurso **não foi encontrado**

// bloco 7 · contrato e erros

O status ajuda a interpretar o resultado da operação

200	201	404	405	422
consulta bem-sucedida	criação	recurso ausente	método não permitido	corpo inválido para o contrato

Leia o status **junto com** o método, o caminho e o corpo da resposta.

4 min para as 10 questões

Quiz 7: questões 1–4

01 Qual código representa a criação válida do alerta?

A) 201

B) 404

C) 405

02 `GET /alertas/999` retorna `404`. Isso significa:

A) JSON sem aspas.

B) alerta solicitado ausente.

C) método proibido em `/metricas`.

03 `POST /alertas` sem `tipo` retorna, neste exemplo:

A) 200

B) 201

C) 422

04 `POST /metricas` encontra o caminho, mas usa método não permitido. Código esperado:

A) 405

B) 404

C) 201

questões 5–7

Quiz 7: questões 5–7

05 O contrato precisa documentar:

A) só a cor da página.

B) método, caminho, entrada e respostas.

C) apenas o nome da linguagem.

06 Ao receber um `404`, podemos afirmar que nenhuma comunicação ocorreu?

A) Sim.

B) Apenas se for GET.

C) Não, o servidor pode ter enviado essa resposta.

07 Qual caso expressa entrada inválida, em vez de recurso ausente?

A) POST sem campo obrigatório.

B) GET de ID 999.

C) GET bem-sucedido.

questões 8–10

Quiz 7: questões 8–10

08 Qual código consultar primeiro ao analisar uma resposta da API?

A) O número da porta como se fosse status.

B) O status HTTP.

C) O ID do slide.

09 Se `GET /metricas` retorna `200`, podemos concluir que:

A) a consulta foi atendida segundo o contrato.

B) todos os pacotes da rede foram analisados.

C) houve criação de alerta.

10 Uma equipe documentou somente casos de sucesso. O que falta ao contrato?

A) A fonte do slide.

B) O nome do navegador.

C) Entradas inválidas e respostas de erro.

Devolutiva do bloco 7

01

`404` = recurso ausente.

02

`405` = método não permitido.

03

`422` = entrada inválida neste exemplo.

// bloco 8 · testes e arquitetura

Uma matriz registra o que esperávamos e o que ocorreu

REQUISIÇÃO	ENTRADA	ESPERADO	OBSERVADO	CONCLUSÃO
`GET /alertas`	sem corpo	`200` + lista JSON	(preencher após o teste)	(preencher)
…	…	…	…	…

esperado

vem do **contrato**, antes do teste

observado

vem da **execução**, depois do teste

// bloco 8 · testes e arquitetura

Resultado observado só pode ser preenchido após o teste

esperado

definido pelo contrato

observado

lido no terminal, no navegador ou no DevTools

evidência preparada pelo professor

Deve ser identificada como **exemplo**, não como medição feita pelo aluno.

// bloco 8 · testes e arquitetura

Os casos positivos verificam consulta e criação

REQUISIÇÃO	ENTRADA	ESPERADO	OBSERVADO
`GET /alertas`	sem corpo	`200`	`[]`
`GET /alertas/1`	ID 1 no caminho	`200`	`[]`
`POST /alertas`	JSON válido	`201`	`[]`

Depois do POST, consulte `GET /alertas` e confirme o alerta de `id: 3`.

// bloco 8 · testes e arquitetura

Os casos negativos verificam limites do contrato

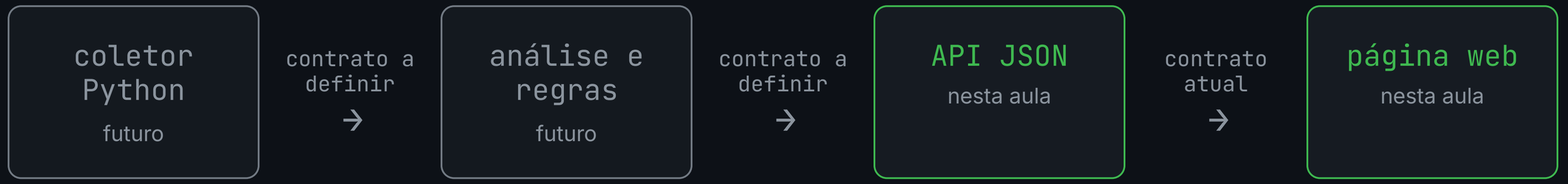
REQUISIÇÃO	ENTRADA	ESPERADO	OBSERVADO
`GET /alertas/999`	ID inexistente	`404`	[ ] _____
`POST /alertas`	JSON sem `tipo`	`422`	[ ] _____
`POST /metricas`	qualquer corpo	`405`	[ ] _____

pense

Cada erro ocorreu pela mesma razão?

// bloco 8 · testes e arquitetura

O IDS terá quatro partes conectadas por interfaces



nesta aula

API com dados ilustrativos e página.

futuro

Captura real e regras de detecção.

// prática · 13 minutos

Não preencha “observado” antes de executar

Prática: teste o contrato e revise o desenho do IDS

01 Execute os três casos positivos e os três negativos.

02 Preencha “resultado observado” na matriz.

03 Compare cada observação ao resultado esperado.

04 Corrija a tabela do contrato se houver divergência real.

05 No diagrama do IDS, marque o que está funcionando hoje.

06 Explique onde entrarão coletor e regras nas aulas futuras.

4 min para as 10 questões

Quiz 8: questões 1–4

01 Qual coluna da matriz deve vir do contrato antes da execução?

A) Resultado esperado.

B) Resultado observado.

C) Conclusão final.

02 Onde registrar o status realmente obtido no teste?

A) No título do arquivo Python.

B) Em resultado observado.

C) No endereço IP ilustrativo.

03 `GET /alertas/999` é um caso útil porque verifica:

A) criação válida.

B) a cor do JSON.

C) recurso inexistente.

04 Qual parte do IDS apresenta os dados ao usuário?

A) Página web.

B) Coletor.

C) Regra de detecção.

Quiz 8: questões 5–7

05 Onde se encaixa uma regra que decide quando gerar alerta?

A) No navegador como responsabilidade obrigatória.

B) No módulo de análise.

C) No rodapé do contrato.

06 A API mostra alertas de exemplo nesta aula. Isso prova captura real?

A) Sim.

B) Se houver dois alertas, sim.

C) Não.

07 Esperado `201`, observado `422` em um POST. Primeira ação:

A) examinar o corpo enviado e os campos exigidos.

B) declarar sucesso.

C) trocar GET por DNS.

Quiz 8: questões 8–10

08 Uma resposta preparada pelo professor pode ser registrada como resultado observado pelo aluno sem identificação?

A) Sim, sempre.

B) Não; sua origem e limitação devem ser indicadas.

C) Apenas se for 200.

09 O que confirma o efeito de um POST bem-sucedido?

A) Um GET posterior que inclui o novo alerta.

B) O fechamento do Chrome.

C) A existência de uma pasta.

10 Qual sequência descreve a arquitetura futura?

A) Página → CSS → slide → impressora.

B) DNS → teclado → cabo → JSON.

C) Coleta → análise/regras → API → página.

Devolutiva do bloco 8

01

O teste compara **esperado** e **observado**.

02

Casos negativos também fazem parte do contrato.

03

A arquitetura separa coleta, decisão, exposição e apresentação.

// fechamento da aula 09

Entrega e fechamento da Aula 09

01 Tabela de casos da API com resultado esperado e resultado observado.

02 Contrato revisado, incluindo entradas, saídas e erros.

03 Diagrama do IDS indicando componentes atuais e futuros.

para pensar

O que precisará mudar para substituir alertas ilustrativos por alertas produzidos a partir de tráfego observado?